

Decision Trees, Cross-Validation & Ensemble Methods

Complete Study Notes — cleaned, corrected, and organized from the class transcript

Topics covered: Decision Tree structure, recursive binary splitting, greedy approach, overfitting, pre-pruning, post-pruning, entropy, information gain, Gini impurity, ID3, C4.5, CART, holdout validation, K-Fold, LOOCV, stratification, hyperparameter tuning, Grid Search, bagging, Random Forest, and boosting.

1. What Is a Decision Tree?

A decision tree is a supervised machine-learning algorithm that makes predictions by repeatedly splitting data according to feature-based conditions. It can be used for both **classification** and **regression**.

Tree terminology:

- **Root node:** the first and topmost decision node.
- **Internal/decision node:** a node that performs another split.
- **Branch:** the outcome of a decision rule.
- **Leaf/terminal node:** the final node that produces a prediction.
- **Parent node:** a node that splits into child nodes.
- **Child node:** a node created from a parent split.

The main goal is to create increasingly homogeneous child nodes. In classification, a perfectly pure node contains observations from only one class.

2. Pure and Impure Nodes

Imagine a container containing red, blue, and green balls. The container is **impure** because multiple classes are mixed together. If the balls are separated until each final group contains only one color, those groups are **pure**.

A decision tree follows the same basic idea: start with mixed observations and select splits that reduce class impurity.

Important correction: a leaf does not have to be perfectly pure. Real trees are often intentionally stopped before perfect purity to avoid overfitting.

3. Recursive Binary Splitting

Many decision-tree algorithms grow a tree through **recursive binary splitting**. At each step, the algorithm searches for a feature and threshold that divide the current node into two child regions.

The same procedure is then repeated recursively on the resulting child nodes until a stopping rule is reached.

Example: $Alcohol \leq 9.6$? may split observations into two groups. Each group can then be split again using acidity, sulfides, or another useful feature.

4. Why Decision Trees Are Called Greedy

Tree-building is generally described as a **greedy** process because the algorithm selects the best split available at the current node according to its split criterion. It does not search every possible future tree and guarantee a globally optimal final structure.

A locally strong split may eventually lead to a larger or less useful tree. Exhaustively evaluating every possible tree would usually be computationally infeasible.

Key idea: greedy means locally optimal at the current step—not that the algorithm ignores all calculations or splits randomly.

5. Why Decision Trees Overfit

An unrestricted tree can continue splitting until it models very small details and noise in the training data. This creates **high variance**: small changes in the training data can produce a substantially different tree.

A very deep tree may achieve excellent training performance but poor performance on unseen data.

Common controls: max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, minimum impurity decrease, cost-complexity pruning, and cross-validation.

6. Tree Pruning

Pruning reduces tree complexity to improve generalization.

Pre-pruning (early stopping): restricts growth while the tree is being built. Examples include limiting maximum depth, requiring a minimum number of samples before splitting, and requiring a minimum number of samples in a leaf.

Post-pruning: first grows a larger tree and then removes branches that do not provide enough validated improvement. Cost-complexity pruning is a standard example.

Important correction: pre-pruning is not universally “better” than post-pruning. Pre-pruning is cheaper, but it can stop useful growth too early. Post-pruning may produce a better subtree but requires more computation.

7. Entropy

Entropy measures class uncertainty or impurity in a classification node. A node containing one class has entropy 0. A more evenly mixed node has higher entropy.

Binary-class formula: $H = -p \log_2(p) - (1-p) \log_2(1-p)$.

For multiple classes, entropy is the negative sum of $p_i \log_2(p_i)$ across all classes.

The tree prefers splits that reduce the weighted impurity of the child nodes.

8. Information Gain

Information gain measures how much entropy is reduced by a split.

Information Gain = Parent Entropy – Weighted Child Entropy

A larger information gain means the split creates purer child nodes relative to the parent.

Important correction: a node is not declared final because information gain becomes “very less” by itself. Stopping depends on the algorithm and configured stopping/pruning rules.

9. Gini Impurity

Gini impurity is another measure of class mixing.

Formula: $Gini = 1 - \sum p_i^2$

A pure node has Gini impurity 0. Higher values indicate greater class mixing.

For a candidate feature/split, the algorithm calculates the weighted impurity of the resulting children. The preferred split is the one with the **largest impurity reduction**, equivalently the smallest weighted child impurity among the candidates being compared.

Gini is computationally simpler than entropy because it avoids logarithms, although in modern practical workloads the speed difference is often small. Gini and entropy frequently produce similar trees, but they are not mathematically identical.

10. Choosing the Root Node

The root is not selected simply by taking a whole column with the smallest raw Gini value. The algorithm evaluates **candidate splits** for features and selects the split that gives the best impurity reduction.

For categorical examples, one can compare weighted impurity after splitting by candidate features. For continuous features, the algorithm evaluates possible thresholds.

Using entropy, select the candidate with the highest information gain. Using Gini, select the candidate that produces the greatest Gini decrease.

11. Decision-Tree Algorithm Families

ID3: historically uses information gain and was designed primarily for categorical classification features.

C4.5: extends ID3 and handles continuous attributes, missing values, and pruning. It is still fundamentally a classification-tree algorithm—not a general regression-tree algorithm.

CART (Classification and Regression Trees): supports both classification and regression and produces binary splits. Classification commonly uses Gini impurity; regression commonly uses squared-error-related criteria.

Important correction: C4.5 should not be described as a regression algorithm merely because it accepts continuous input features.

12. Decision Tree Advantages and Disadvantages

Advantages:

- Works for classification and regression.
- Easy to visualize and explain when reasonably small.
- Captures nonlinear relationships and feature interactions.
- Usually does not require feature scaling.
- Can work with mixed feature behavior after appropriate preprocessing.

Disadvantages:

- High risk of overfitting without controls.
- High variance and instability.
- Greedy construction does not guarantee the globally optimal tree.
- Deep trees can become difficult to interpret.
- Some implementations require categorical variables to be encoded.

13. Holdout Method

The holdout method divides the dataset into separate subsets, commonly training and testing sets. The model is fitted on training data and evaluated on unseen test data.

It is simple and computationally efficient, but the measured performance can depend strongly on the particular random split.

Best practice: when model selection or hyperparameter tuning is performed, use training/validation/test separation or cross-validation rather than repeatedly tuning against the final test set.

14. K-Fold Cross-Validation

In K-Fold cross-validation, the data is divided into K folds. The model is trained K times. Each fold serves once as the validation fold while the remaining $K-1$ folds are used for training.

The K validation scores are then summarized, usually using the mean and often the standard deviation.

Advantages: more robust performance estimation than a single split and better use of limited data.

Disadvantage: greater computational cost because the model is fitted multiple times.

15. Leave-One-Out Cross-Validation (LOOCV)

LOOCV is the special case of K-Fold where K equals the number of observations. Each iteration holds out exactly one observation and trains on all remaining observations.

It uses almost all available data for training in every iteration, but it can be extremely computationally expensive for large datasets.

Practical point: LOOCV is not automatically more accurate or preferable than ordinary K-Fold. It can have high variance as an estimator and is rarely the default choice for large modern datasets.

16. Stratified Splitting and Stratified K-Fold

Stratification preserves approximately the same class proportions across splits.

Example: if the original classification dataset contains 60% class A and 40% class B, a stratified training and test split attempts to preserve roughly that 60/40 ratio in both subsets.

This is particularly useful for classification datasets with unequal class distributions.

Important correction: stratification preserves class proportions; it does not itself solve class imbalance.

Techniques such as class weighting, resampling, threshold tuning, and appropriate evaluation metrics may still be required.

17. Decision Tree Hyperparameters

Important hyperparameters include:

- **criterion:** split-quality measure, such as Gini or entropy/log-loss depending on implementation.
- **splitter:** strategy for selecting candidate splits, such as best or random.
- **max_depth:** maximum tree depth.
- **min_samples_split:** minimum observations needed to split an internal node.
- **min_samples_leaf:** minimum observations required in a leaf.
- **max_leaf_nodes:** optional limit on terminal nodes.
- **ccp_alpha:** cost-complexity pruning strength in scikit-learn.

Hyperparameters are configured before/during model training; they are not learned in the same way as the tree's split structure.

18. Grid Search for Hyperparameter Tuning

Grid Search evaluates combinations from a predefined hyperparameter grid. For example, it may test multiple values of criterion, maximum depth, minimum samples per leaf, and splitter strategy.

The best combination is selected according to a chosen cross-validation scoring metric.

Critical point: Grid Search only finds the best combination *within the supplied search grid*. It does not guarantee the globally best possible model.

A very large grid can become computationally expensive. Randomized Search, Bayesian optimization, and specialized optimization frameworks may be more efficient for large search spaces.

19. Standardization and PCA: Are They Needed for Trees?

Decision trees generally **do not require standardization or normalization** because splits depend on thresholds and ordering rather than Euclidean distance.

PCA is also not a standard requirement for decision trees. It may sometimes be used for dimensionality reduction, but it can reduce interpretability and is not automatically beneficial.

Practical rule: do not add StandardScaler or PCA to a tree pipeline without a specific reason and validation evidence.

20. Ensemble Learning

Ensemble learning combines multiple models to produce a stronger overall predictor.

Two major families discussed here are **bagging** and **boosting**.

The purpose is not simply to connect trees visually. The models use different training strategies to reduce error and improve generalization.

21. Bagging

Bagging stands for **Bootstrap Aggregating**. Multiple models are trained on bootstrap samples—samples drawn from the training data with replacement.

The models can be trained independently and therefore often in parallel.

For classification, predictions may be combined through majority voting or averaged class probabilities. For regression, predictions are usually averaged.

Bagging primarily helps reduce **variance**.

22. Random Forest

A Random Forest is a bagging-based ensemble of decision trees with an additional source of randomness: at each split, only a random subset of features is considered.

This feature randomness helps decorrelate the trees. Combining many less-correlated trees usually produces a more stable model than a single deep decision tree.

Important correction: the dataset is not simply divided into non-overlapping pieces where each tree receives one unique part. Standard Random Forest trees are typically trained on separate **bootstrap samples**, which can overlap.

For classification, the forest aggregates class predictions/probabilities. For regression, it averages numerical predictions.

23. Boosting

Boosting builds weak learners **sequentially**. Later learners focus on correcting mistakes or residual errors from the current ensemble.

Major families include:

- **AdaBoost:** increases emphasis on previously misclassified observations.
- **Gradient Boosting:** sequentially fits learners to reduce a differentiable loss, often by fitting residual-like targets.
- **XGBoost:** an optimized regularized gradient-boosting framework.

Important correction: boosting did not arise simply because parallel bagging models had to be “fixed individually.” Bagging and boosting address error differently: bagging mainly reduces variance through averaging, while boosting sequentially reduces the ensemble's remaining errors and can reduce bias.

24. Bagging vs Boosting

Bagging: models are largely independent; training can be parallel; uses bootstrap sampling; combines predictions; mainly reduces variance; Random Forest is the classic example.

Boosting: models are sequentially dependent; each stage responds to the current ensemble's errors; often reduces bias and can also reduce variance; examples include AdaBoost, Gradient Boosting, and XGBoost.

Neither approach is universally superior. The right choice depends on data size, noise, interpretability, computational constraints, and validation performance.

25. End-to-End Practical Workflow

1. Define whether the problem is classification or regression.
2. Clean and understand the dataset.
3. Split data correctly; stratify classification targets when appropriate.
4. Build a simple baseline decision tree.
5. Evaluate with suitable metrics.
6. Control complexity using pruning-related hyperparameters.
7. Use cross-validation for robust model selection.
8. Tune a focused hyperparameter search.
9. Compare against ensemble methods such as Random Forest and boosting.
10. Evaluate the final selected model once on untouched test data.

26. Interview-Ready Answers

Why does a decision tree overfit? An unrestricted tree can keep splitting until it learns training-specific noise, producing high variance and poor generalization.

Entropy vs Gini? Both measure node impurity. Entropy uses logarithms; Gini uses squared class probabilities. They often produce similar results.

What is information gain? The reduction in entropy produced by a split.

What is pruning? Restricting or removing unnecessary tree growth to reduce complexity and improve generalization.

What is K-Fold cross-validation? Repeatedly training and validating across K different folds so every fold is used once for validation.

What is Random Forest? An ensemble of decision trees trained using bootstrap samples and random feature subsets, with predictions aggregated across trees.

Bagging vs boosting? Bagging trains independent learners and aggregates them; boosting trains learners sequentially so later learners improve the current ensemble.

Quick Comparison Tables

Concept	Main Purpose	Preferred Direction
Entropy	Measure class impurity	Lower is purer
Information Gain	Measure entropy reduction	Higher is better
Gini Impurity	Measure class impurity	Lower is purer
Tree Depth	Control model complexity	Tune with validation

Method	Training Style	Primary Benefit	Example
Single Tree	One greedy tree	Interpretability	Decision Tree
Bagging	Independent / parallelizable	Reduce variance	Random Forest
Boosting	Sequential	Correct ensemble errors	XGBoost

Recommended study order: Decision Tree structure → recursive splitting → greedy behavior → overfitting → pruning → entropy → information gain → Gini → ID3/C4.5/CART → holdout → K-Fold → LOOCV → stratification → hyperparameters → Grid Search → bagging → Random Forest → boosting.